

Migrate from Pages to Workers

You can deploy full-stack applications, including front-end static assets and back-end APIs, as well as server-side

[Cloudflare Workers](#).

Like Pages, requests for static assets on Workers are free, and Pages Functions invocations are charged at the same rate as Workers, so you can expect [a similar cost structure](#).

Unlike Pages, Workers has a distinctly broader set of features available to it, (including Durable Objects, Cron Triggers, and more comprehensive Observability). A complete list can be found at [the bottom of this page](#). Workers will receive the focus of Cloudflare's development efforts going forwards, so we therefore [are recommending using Cloudflare Workers over Cloudflare Pages for any new projects ↗](#).

Migration guide

Migrating from Cloudflare Pages to Cloudflare Workers is often a straightforward process. The following are some of the most common steps you will need to take to migrate your project.

Frameworks

If your Pages project uses [a popular framework](#), most frameworks already have adaptors available for Cloudflare Workers. Switch out any Pages-specific adaptors for the Workers equivalent and follow any guidance that they provide.

Project configuration

If your project doesn't already have one, create a [Wrangler configuration file](#) (either `wrangler.jsonc`, `wrangler.json` or `wrangler.toml`) in the root of your project. The two mandatory fields are:

- [name](#)

Set this to the name of the Worker you wish to deploy to. This can be the same as your existing Pages project name, so long as it conforms to Workers' name restrictions (e.g. max length).

- [compatibility_date](#).

If you were already using [Pages Functions](#), set this to the same date configured there. Otherwise, set it to the current date.

Build output directory

Where you previously would configure a "build output directory" for Pages (in either a [Wrangler configuration file](#) or in [the Cloudflare dashboard](#)), you must now set the [assets.directory](#) value for a Worker project.

Before, with **Cloudflare Pages**:

`{}` `wrangler.jsonc`  `wrangler.toml`

```
name = "my-pages-project"
pages_build_output_dir = "./dist/client/"
```

Now, with **Cloudflare Workers**:

-  **Copy Page as Markdown**
Copy the raw Markdown content to clipbo
-  **View Page as Markdown**
Open the Markdown file in a new tab

`{}` wrangler.jsonc  **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"

[assets]
directory = "./dist/client/"
```

Serving behavior

Pages would automatically attempt to determine the type of project you deployed. It would look for `404.html` and `index.html` files as signals for whether the project was likely a [Single Page Application \(SPA\)](#) or if it should [serve custom 404 pages](#).

In Workers, to prevent accidental misconfiguration, this behavior is explicit and [must be set up manually](#).

For a Single Page Application (SPA):

`{}` wrangler.jsonc  **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"

[assets]
directory = "./dist/client/"
not_found_handling = "single-page-application"
```

For custom 404 pages:

`{}` wrangler.jsonc  **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"

[assets]
directory = "./dist/client/"
not_found_handling = "404-page"
```

Ignoring assets

Pages would automatically exclude some files and folders from being uploaded as static assets such as `node_modules`, `.DS_Store`, and `.git`. If you wish to also avoid uploading these files to Workers, you can create an [.assetsignore file](#) in your project's static asset directory.

`dist/client/assetsignore`

```
*/node_modules
*/.DS_Store
*/.git
```

Pages Functions

Full-stack framework

If you use a full-stack framework powered by Pages Functions, ensure you have [updated your framework](#) to target Workers instead of Pages.

Pages Functions with an "advanced mode" `_worker.js` file

If you use Pages Functions with an ["advanced mode" `worker.js` file](#), you must first ensure this script doesn't get uploaded as a static asset. Either move `_worker.js` out of the static asset directory (recommended), or create an [.assetsignore file](#) in the static asset directory and include `_worker.js` within it.

```
dist/client/assetsignore
```

```
_worker.js
```

Then, update your configuration file's main field to point to the location of this Worker script:

`{}` wrangler.jsonc  `wrangler.toml`

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./dist/client/_worker.js"

[assets]
directory = "./dist/client/"
```

Pages Functions with a `functions/` folder

If you use **Pages Functions with a [folder of functions/](#)**, you must first compile these functions into a single Worker script with the [wrangler pages functions build](#) command.

 `npm`  `yarn`  `pnpm`

```
npx wrangler pages functions build --outdir=./dist/worker/
```

Although this command will remain available to you to run at any time, we do recommend considering using another framework if you wish to continue to use file-based routing. [HonoX](#) is one popular option.

Once the Worker script has been compiled, you can update your configuration file's main field to point to the location it was built to:

`{}` wrangler.jsonc  `wrangler.toml`

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./dist/worker/index.js"

[assets]
directory = "./dist/client/"
```

`_routes.json` and Pages Functions middleware

If you authored a [_routes.json](#) file in your Pages project, or used [middleware](#) in Pages Functions, you must pay close attention to the configuration of your Worker script. Pages would default to serving your Pages Functions ahead of static assets and `_routes.json` and Pages Functions middleware allowed you to customize this behavior.

Workers, on the other hand, will default to serving static assets ahead of your Worker script, unless you have configured [assets.run_worker_first](#). This option is required if you are, for example, performing any authentication checks or logging requests before serving static assets.

`{}` wrangler.jsonc `⚙️` **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./dist/worker/index.js"

[assets]
directory = "./dist/client/"
run_worker_first = true
```

Starting from scratch

If you wish to, you can start a new Worker script from scratch and take advantage of all of Wrangler's and the latest runtime features (e.g. [WorkerEntrypoints](#), [TypeScript support](#), [bundling](#), etc.):

JS JavaScript **TS** TypeScript

`./worker/index.js`

```
import { WorkerEntrypoint } from "cloudflare:workers";

export default class extends WorkerEntrypoint {
  async fetch(request) {
    return new Response("Hello, world!");
  }
}
```

`{}` wrangler.jsonc `⚙️` **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./worker/index.ts"

[assets]
directory = "./dist/client/"
```

Assets binding

Pages automatically provided [an ASSETS binding](#) to access static assets from Pages Functions. In Workers, the name of this binding is customizable and it must be manually configured:

`{}` wrangler.jsonc `⚙️` **wrangler.toml**

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./worker/index.ts"

[assets]
directory = "./dist/client/"
binding = "ASSETS"
```

Runtime

If you had customized [placement](#), or set a [compatibility_date](#) or any [compatibility_flags](#) in your Pages project, you can define the same in your Wrangler configuration file:

{ } wrangler.jsonc

⚙️ wrangler.toml

```
name = "my-worker"
compatibility_date = "2025-04-01"
compatibility_flags = [ "nodejs_compat" ]
main = "./worker/index.ts"

[placement]
mode = "smart"

[assets]
directory = "./dist/client/"
binding = "ASSETS"
```

Variables, secrets and bindings

[Variables](#) and [bindings](#) can be set in your [Wrangler configuration file](#) and are made available in your Worker's environment (env). [Secrets](#) can be uploaded with Wrangler or defined in the Cloudflare dashboard for [production](#) and [.dev.vars for local development](#).

If you are [using Workers Builds](#), ensure you also [configure any variables relevant to the build environment there](#). Unlike Pages, Workers does not share the same set of runtime and build-time variables.

Wrangler commands

Where previously you used [wrangler_pages_dev](#) and [wrangler_pages_deploy](#), now instead use [wrangler_dev](#) and [wrangler_deploy](#). Additionally, if you are using a Vite-powered framework, [our new Vite plugin](#) may be able to offer you an even simpler development experience.

Builds

If you are using Pages' built-in CI/CD system, you can swap this for Workers Builds by first [connecting your repository to Workers Builds](#) and then [disabling automatic deployments on your Pages project](#).

Preview environment

Pages automatically creates a preview environment for each project, and can be independently configured.

To get a similar experience in Workers, you must:

1. Ensure [preview URLs](#) are enabled (they are on by default).

{ } wrangler.jsonc

⚙️ wrangler.toml

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./worker/index.ts"
preview_urls = true

[assets]
directory = "./dist/client/"
```

2. [Enable non-production branch builds](#) in Workers Builds.

Optionally, you can also [protect these preview URLs with Cloudflare Access](#).

Note

Unlike Pages, Workers does not natively support defining different bindings in production vs. non-production builds. This is something we are actively exploring, but in the meantime, you may wish to consider using [Wrangler Environments](#) and an [appropriate Workers Build configuration](#) to achieve this.

Headers and redirects

[_headers](#) and [_redirects](#) files are supported natively in Workers with static assets. Ensure that, just like for Pages, these files are included in the static asset directory of your project.

pages.dev

Where previously you were offered a `pages.dev` subdomain for your Pages project, you can now configure a personalized `workers.dev` subdomain for all of your Worker projects. You can [configure this subdomain in the Cloudflare dashboard](#), and opt-in to using it with the [workers_dev](#) option in your configuration file.

`{}` wrangler.jsonc `⚙️` wrangler.toml

```
name = "my-worker"
compatibility_date = "2025-04-01"
main = "./worker/index.ts"
workers_dev = true
```

Custom domains

If your domain's nameservers are managed by Cloudflare, you can, like Pages, configure a [custom domain](#) for your Worker. Additionally, you can also configure a [route](#) if you only wish to some subset of paths to be served by your Worker.

Note

Unlike Pages, Workers does not support any domain whose nameservers are not managed by Cloudflare.

Rollout

Once you have validated the behavior of Worker, and are satisfied with the development workflows, and have migrated all of your production traffic, you can delete your Pages project in the Cloudflare dashboard or with Wrangler:

`npm` `yarn` `pnpm`

```
npx wrangler pages project delete
```

Compatibility matrix

This compatibility matrix compares the features of Workers and Pages. Unless otherwise stated below, what works in Pages works in Workers, and what works in Workers works in Pages. Think something is missing from this list? [Open a pull request](#) or [create a GitHub issue](#).

Legend

- ✅ : Supported
- 🕒 : Coming soon
- 🟡 : Unsupported, workaround available
- ❌ : Unsupported

	Workers	Pages
Writing, Testing, and Deploying Code		
Cloudflare Vite plugin	✓	✗
Rollbacks	✓	✓
Gradual Deployments	✓	✗
Preview URLs	✓	✓
Testing tools	✓	✓
Local Development	✓	✓
Remote Development (--remote)	✓	✗
Quick Editor in Dashboard ↗	✓	✗
Static Assets		
Early Hints	✗	✓
Custom HTTP headers for static assets	✓	✓
Middleware	✓ ¹	✓
Redirects	✓	✓
Smart Placement	✓	✓
Serve assets on a path	✓	✗
Observability		
Workers Logs	✓	✗
Logpush	✓	✗
Tail Workers	✓	✗
Real-time logs	✓	✓
Source Maps	✓	✗
Runtime APIs & Compute Models		
Node.js Compatibility Mode	✓	✓
Durable Objects	✓	🟡 ²
Cron Triggers	✓	✗
Bindings		
AI	✓	✓
Analytics Engine	✓	✓
Assets	✓	✓
Browser Rendering	✓	✓

	Workers	Pages
D1	✓	✓
Email Workers	✓	✗
Environment Variables	✓	✓
Hyperdrive	✓	✓
Image Resizing	✓	✗
KV	✓	✓
mTLS	✓	✓
Queue Producers	✓	✓
Queue Consumers	✓	✗
R2	✓	✓
Rate Limiting	✓	✗
Secrets	✓	✓
Service bindings	✓	✓
Vectorize	✓	✓
Builds (CI/CD)		
Monorepos	✓	✓
Build Watch Paths	✓	✓
Build Caching	✓	✓
Deploy Hooks	⌚	✓
Branch Deploy Controls	🟡 3	✓
Custom Branch Aliases	⌚	✓
Pages Functions		
File-based Routing	🟡 4	✓
Pages Plugins	🟡 5	✓
Domain Configuration		
Custom domains	✓	✓
Custom subdomains	✓	✓
Custom domains outside Cloudflare zones	✗	✓
Non-root routes	✓	✗

Was this helpful?



[← Billing and Limitations](#)

[Next](#)
[Migrate from Netlify to Workers →](#)

Last updated: Jun 5, 2025

Resources

- API
- New to Cloudflare?
- Products
- Sponsorships
- Open Source

Company

- cloudflare.com
- Our team
- Careers

Community

-  X
-  Discord
-  YouTube
-  GitHub

Support

- Help Center
- System Status
- Compliance
- GDPR

Tools

- Cloudflare Radar
- Speed Test
- Is BGP Safe Yet?
- RPKI Toolkit
- Certificate Transparency